EXPERIMENT 12

AIM:

To detect outliers in a given dataset using the Interquartile Range (IQR) method.

CODE:

```
import numpy as np

import pandas as pd

import seaborn as sns

import matplotlib.pyplot as plt


# Load the dataset

df = pd.read_csv("student_marks_outliers.csv")

print(df)


# Extract Marks as an array

array = df["Marks"].values

print(array)


# Calculate mean

print("Mean:", array.mean())


# Calculate percentiles

print("25th Percentile:", np.percentile(array, 25))

print("50th Percentile:", np.percentile(array, 50))

print("75th Percentile:", np.percentile(array, 75))
```

```
print("100th Percentile:", np.percentile(array, 100))

# Calculate Q1, Q3 and IQR
Q1 = np.percentile(array, 25)
Q3 = np.percentile(array, 75)
IQR = Q3 - Q1

print("Q1 =", Q1)
print("Q3 =", Q3)
print("IQR =", IQR)

# Calculate lower and upper limits
lower_limit = Q1 - (1.5 * IQR)
upper_limit = Q3 + (1.5 * IQR)

print("Lower Limit:", lower_limit)
print("Upper Limit:", upper_limit)

# Detect outliers
outliers = array[
    (array < lower_limit) | (array > upper_limit)
]

print("Outliers:")
print(outliers)
```

```
# Define outlier detection function
def outDetection(array):
    Q1 = np.percentile(array, 25)
    Q3 = np.percentile(array, 75)
    IQR = Q3 - Q1

    lr = Q1 - (1.5 * IQR)
    ur = Q3 + (1.5 * IQR)

    return lr, ur

# Get lower and upper ranges
lr, ur = outDetection(array)

print("Lower Range:", lr)
print("Upper Range:", ur)

# Plot distribution before removing outliers
sns.histplot(array, bins=8, kde=True)
plt.title("Distribution of Student Marks")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.show()
```

```
# Remove outliers
new_array = array[
    (array >= lr) & (array <= ur)
]

print("Data after removing outliers:")
print(new_array)

# Plot distribution after removing outliers
sns.histplot(new_array, bins=8, kde=True)
plt.title("Distribution After Removing Outliers")
plt.xlabel("Marks")
plt.ylabel("Count")
plt.show()
```

OUTPUT:

1. Original Student Marks Dataset

Output of reading student_marks_outliers.csv:

Index	Marks
0	72
1	68
2	75
3	81
4	77
5	69
6	85
7	74
8	80
9	78
10	83
11	71
12	76
13	88
14	79
15	82
16	73
17	86
18	90
19	84
20	67
21	70
22	92
23	87
24	89
25	94
26	66
27	91
28	95
29	76
30	10
31	15
32	130

2. Marks Array and Mean

Output of array.mean():

```
np.float64(77.36363636363636)
```

The marks were stored as a NumPy array. The separate full array printout is omitted because it repeats the dataset above.

3. Percentiles

Percentile	Output
25th Percentile	72.0
50th Percentile	79.0
75th Percentile	87.0
100th Percentile	130.0

4. IQR Calculation

Measure	Output
Q1	72.0
Q3	87.0
IQR = Q3 − Q1	15.0

5. Lower and Upper Limits

Limit	Output
Lower Limit	49.5
Upper Limit	109.5

6. Detected Outliers

Output:

Outliers:
[10 15 130]

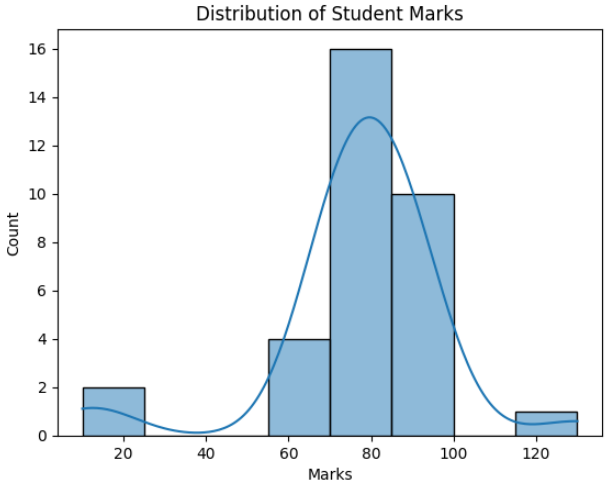
7. User-Defined Outlier Detection Function

The function outDetection(array) returns the lower and upper ranges. Output:

Lower Range: 49.5
Upper Range: 109.5

8. Distribution of Student Marks Before Removing Outliers

Histogram with KDE:



9. Marks After Removing Outliers

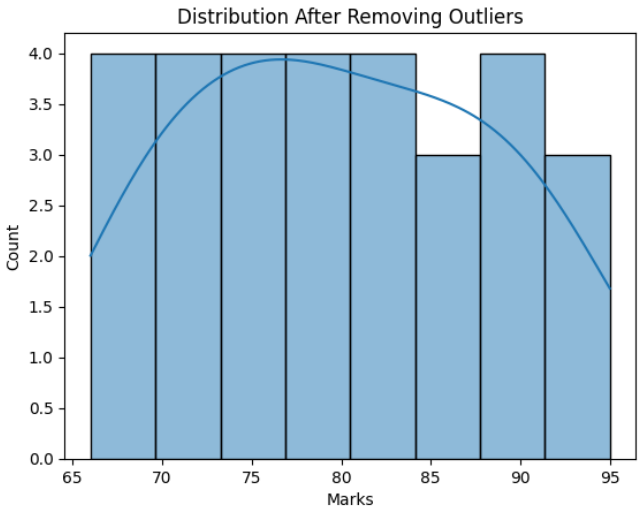
The values within the lower and upper limits were retained:

array([72, 68, 75, 81, 77, 69, 85, 74, 80, 78, 83, 71, 76, 88,
       79, 82, 73, 86, 90, 84, 67, 70, 92, 87, 89, 94, 66, 91, 95, 76])

The outlier values 10, 15, and 130 are excluded from the cleaned array.

10. Distribution After Removing Outliers

Histogram with KDE after outlier removal:



RESULT:

Outliers in the dataset were successfully identified using the IQR method, and the effect of removing them was visualised.